

Good practices for requirements engineering

"Welcome to the group, Sarah," said the project manager, Kristin. "We're looking forward to having you help us with the requirements for this project. I understand that you were a business analyst in your previous job. Do you have some idea of how we should get started here?"

"Well," Sarah replied, "I was thinking I should just interview some users and see what they want. Then I'll write up what they tell me. That should give the developers a good place to start. That's mostly what we did before. Do you know some users I could talk to?"

"Hmmm. Do you think that will be good enough for this type of project?" Kristin asked. "We tried that approach before, but it didn't work out very well. I was hoping you might have some ideas about best practices from your past BA experiences that might be better than just interviewing a couple of users. Are there any particular techniques that you've found to be especially helpful?"

Sarah was rather at a loss. "I don't really know about any specific ways to approach requirements other than talking to users and trying to write clear specifications from what they say. At my last job I just did the best I could based on my business experience. Let me see what I can find out."

Every software professional needs to acquire a tool kit of techniques she can use to approach each project challenge. A practitioner who lacks such a tool kit is forced to invent an approach based on whatever seems reasonable at the moment. Such ad hoc methods rarely yield great results. Some people advocate for specific software development methodologies, packaged sets of techniques that purport to provide holistic solutions to your project challenges. However, simply following a script—a standard process that's supposed to work in every situation—doesn't work very well, either. We find it more effective to identify and apply industry best practices. The best-practice approach stocks your software tool kit with a variety of techniques you can apply to diverse problems.

The notion of best practices is debatable: who decides what is "best" and on what basis? One approach is to convene a body of industry experts to analyze projects from many organizations. These experts seek out practices whose effective performance is associated with successful projects and which are performed poorly or not at all on failed projects. Through these means, the experts reach consensus on the activities that consistently yield superior results and label them *best practices*.

Table 3-1 lists more than 50 practices, grouped into 7 categories, that can help all development teams do a better job on their requirements activities. Several of the practices contribute to more than one category, but each practice appears only once in the table. Most of these practices

contribute to more effective communication among project stakeholders. Note that this chapter is titled “Good practices for requirements engineering,” not “Best practices.” It’s doubtful whether all of these practices will ever be systematically evaluated for this purpose. Nonetheless, many other practitioners have found these techniques to be effective (Sommerville and Sawyer 1997; Hofmann and Lehner 2001; Gottesdiener 2005; IIBA 2009).

TABLE 3-1 Requirements engineering good practices

Elicitation	Analysis	Specification	Validation
■ Define vision and scope ■ Identify user classes ■ Select product champions ■ Conduct focus groups ■ Identify user requirements ■ Identify system events and responses ■ Hold elicitation interviews ■ Hold facilitated elicitation workshops ■ Observe users performing their jobs ■ Distribute questionnaires ■ Perform document analysis ■ Examine problem reports ■ Reuse existing requirements	■ Model the application environment ■ Create prototypes ■ Analyze feasibility ■ Prioritize requirements ■ Create a data dictionary ■ Model the requirements ■ Analyze interfaces ■ Allocate requirements to subsystems	■ Adopt requirement document templates ■ Identify requirement origins ■ Uniquely label each requirement ■ Record business rules ■ Specify nonfunctional requirements	■ Review the requirements ■ Test the requirements ■ Define acceptance criteria ■ Simulate the requirements
Requirements management	Knowledge	Project management	
■ Establish a change control process ■ Perform change impact analysis ■ Establish baselines and control versions of requirements sets ■ Maintain change history ■ Track requirements status ■ Track requirements issues ■ Maintain a requirements traceability matrix ■ Use a requirements management tool	■ Train business analysts ■ Educate stakeholders about requirements ■ Educate developers about application domain ■ Define a requirements engineering process ■ Create a glossary	■ Select an appropriate life cycle ■ Plan requirements approach ■ Estimate requirements effort ■ Base plans on requirements ■ Identify requirements decision makers ■ Renegotiate commitments ■ Manage requirements risks ■ Track requirements effort ■ Review past lessons learned	

This chapter describes each good practice briefly and provides references to other chapters in this book or to other sources where you can learn more about the technique. These practices aren’t suitable for every situation, so use good judgment, common sense, and experience. Even the best practices need to be selected, applied, and adapted thoughtfully to appropriate situations by skilled business analysts. Different practices might be most appropriate for understanding the requirements for different portions of a given project. Use cases and user interface prototypes might help for the client side, whereas interface analysis is more valuable on the server side, for example.

The people who perform or take a lead role in these practices will vary from practice to practice and from project to project. The business analyst (BA) will play a major role with many of them, but not every project has a BA. The product owner could perform some of the practices on an agile project. Still other practices are the purview of the project manager. Think about who the right people in your team are to lead or participate in the practices you select for your next project.



Important None of these techniques will work if you're dealing with unreasonable people. Customers, managers, and IT people sometimes appear to be unreasonable, but perhaps they are just uninformed. They might not know why you want to use certain practices and could be uncomfortable with unfamiliar terms and activities. Try educating your collaborators about the practices, why you want to use them, and why it is important to their own goals to cooperate.

A requirements development process framework

As you saw in Chapter 1, “The essential software requirement,” requirements development involves elicitation, analysis, specification, and validation. Don't expect to perform these activities in a simple linear, one-pass sequence, though. In practice, these activities are interwoven, incremental, and iterative, as shown in Figure 3-1. “Progressive refinement of detail” is a key operating phrase for requirements development, moving from initial concepts of what is needed toward further precision of understanding and expression.

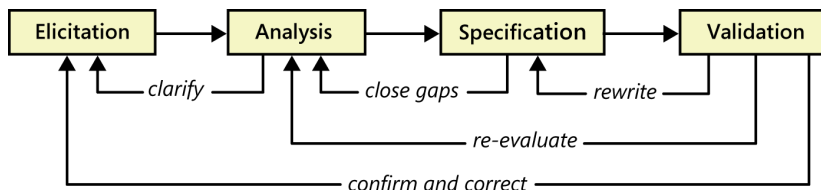


FIGURE 3-1 Requirements development is an iterative process.

If you're the BA, you'll be asking customers questions, listening to what they say, and watching what they do (elicitation). You'll process this information to understand it, classify it in various categories, and relate the customer needs to possible software requirements (analysis). Your analysis might lead you to realize that you need to clarify some requirements, so you go back and do more elicitation. You'll then structure the customer input and derived requirements as written requirement statements and diagrams (specification). While writing requirements, you might need to go back and do some additional analysis to close gaps in your knowledge. Next, you'll ask some stakeholders to confirm that what you've captured is accurate and complete and to correct any errors (validation). You'll do all this for the set of requirements that are most important and most timely for beginning software development. Validation could lead you to rewrite some unclear requirements, revisit some of your analysis activities, or even have to go back and perform additional elicitation. Then you'll move on to the next portion of the project and do it all again. This iterative process continues throughout requirements development and possibly—as with agile projects—throughout the full project duration.

Because of the diversity of software development projects and organizational cultures, there is no single, formulaic approach to requirements development. Figure 3-2 suggests a process framework for requirements development that will work, with sensible adjustments, for many projects. The business need or market opportunity is the predecessor for the process shown in Figure 3-2. These steps are generally performed approximately in numerical sequence, but the process is not strictly sequential. The first seven steps are typically performed once early in the project (although the team will need to revisit all of these activities periodically). The remaining steps are performed for each release or development iteration. Many of these activities can be performed iteratively, and they can be interwoven. For instance, you can perform steps 8, 9, and 10 in small chunks, performing a review (step 12) after each iteration.

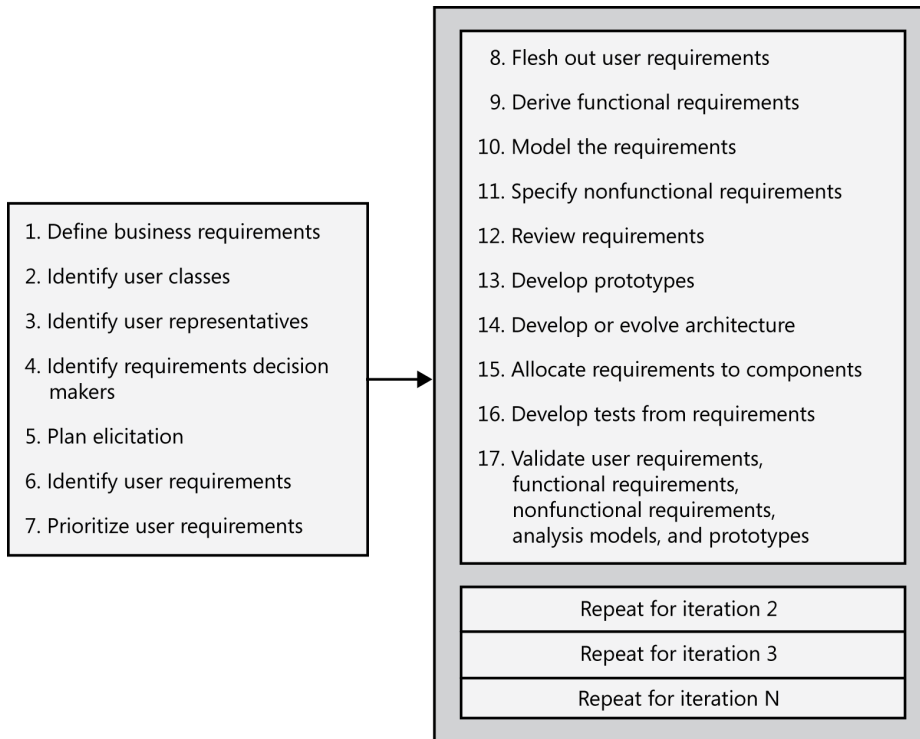


FIGURE 3-2 A representative requirements development process.

The fifth subdiscipline of requirements engineering is requirements management. Requirements management encompasses practices that help you deal with requirements after you have them in hand. These practices include version control and baselining, change control, tracking requirements status, and tracing requirements to other system elements. Requirements management will take place throughout the project's duration at a low level of intensity.

Figure 3-3 illustrates how some common software development life cycles allocate requirements effort across the product development period. The total requirements effort might not be much different for projects of comparable size that follow different life cycles, but the timing distribution of requirements work is very different. In the pure waterfall life cycle, you plan to do only one major

release, so most of the requirements development effort is allocated for the beginning of the project (the solid line in Figure 3-3). This approach is still used on quite a few projects, and it is appropriate for some. But even if you plan a traditional “requirements phase” at the beginning of the project that then leads into design, you can count on having to do some additional requirements work throughout the project.

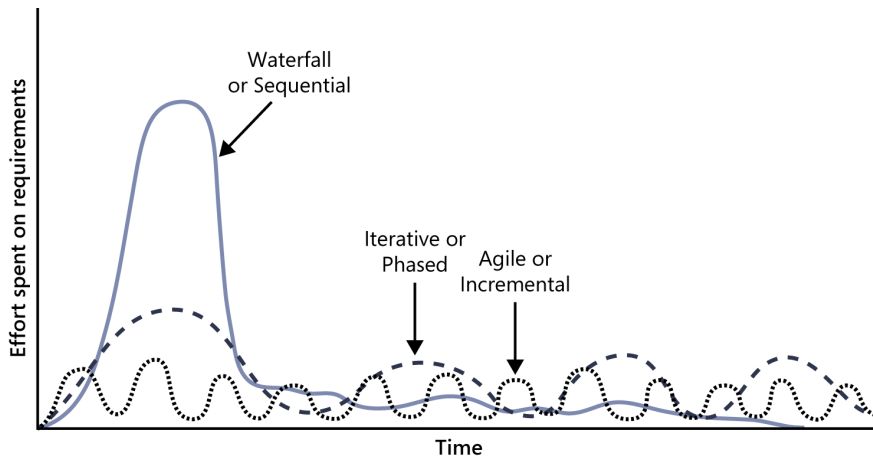


FIGURE 3-3 The distribution of requirements development effort over time varies for projects that follow different development life cycles.

Projects that follow an iterative development process, such as the Rational Unified Process (Jacobson, Booch, and Rumbaugh 1999), will work on requirements on every iteration through the development process, with a heavier emphasis in the first iteration (the dashed line in Figure 3-3). This is also the case if you are planning a series of phased releases, each of which delivers a significant fraction of the product’s ultimate functionality.

Agile and other incremental development projects aim to release functionality every few weeks (Larman 2004). They will have frequent but small requirements development efforts, as shown with the dotted line in Figure 3-3. Such projects begin by doing a first cut at collecting user requirements in the form of simple user stories that describe major objectives the user wants to accomplish with the help of the system. In this approach, you need to learn enough about the stories so that you can estimate their development effort and prioritize them. Prioritizing these user requirements lets you determine which ones to allocate to specific development increments, called iterations or sprints. Those allocated requirements can be explored in further detail in a just-in-time fashion for each development cycle.

Regardless of the life cycle your project follows, you should ask yourself for each release or iteration which of the activities shown in Figure 3-2 will add value and reduce risk. After you have completed step 17 for any portion of the requirements, you’re ready to commence construction of that part of the system. Repeat steps 8 through 17 with the next set of user requirements, which will lay the foundation for the subsequent release or increment.

Good practices: Requirements elicitation

Chapter 1 discussed the three levels of requirements: business, user, and functional. These come from different sources at different times during the project, have different audiences and purposes, and need to be documented in different ways. You also need to elicit nonfunctional requirements, such as quality expectations in various dimensions, from appropriate sources. Following are some practices that can help with eliciting the myriad types of requirements information.

Define product vision and project scope The vision and scope document contains the product's business requirements. The vision statement gives all stakeholders a common understanding of the product's outcome. The scope defines the boundary between what's in and what's out for a specific release or iteration. Together, the vision and scope provide a reference against which to evaluate proposed requirements. The vision should remain relatively stable throughout the project, but each planned release or iteration needs its own scope statement. See Chapter 5, "Establishing the business requirements," for more information.

Identify user classes and their characteristics To avoid overlooking the needs of any user community, identify the various groups of users for your product. They might differ in frequency of use, features used, privilege levels, or experience. Describe aspects of their job tasks, attitudes, location, or personal characteristics that might influence product design. Create user personas, descriptions of imaginary people who will represent particular user classes. See Chapter 6, "Finding the voice of the user," for more information.

Select a product champion for each user class Identify an individual who can accurately serve as the literal voice of the customer for each user class. The product champion presents the needs of the user class and makes decisions on its behalf. This is easiest for internal information systems development, where your users are fellow employees. For commercial product development, build on your current relationships with major customers or beta test sites to locate appropriate product champions. See Chapter 6 for more information.

Conduct focus groups with typical users Convene groups of representative users of your previous products or of similar products. Collect their input on both functionality and quality characteristics for the product under development. Focus groups are particularly valuable for commercial product development, for which you might have a large and diverse customer base. Unlike product champions, focus groups generally do not have decision-making authority. See Chapter 7, "Requirements elicitation," for more information.

Work with user representatives to identify user requirements Explore with your user representatives the tasks they need to accomplish with the software and the value they're trying to achieve. User requirements can be expressed in the form of use cases, user stories, or scenarios. Discuss the interactions between the users and the system that will allow them to complete each task. See Chapter 8, "Understanding user requirements," for more information.

Identify system events and responses List the external events that the system can experience and its expected response to each event. There are three classes of external events. Signal events are control signals or data received from external hardware devices. Temporal, or time-based, events

trigger a response, such as an external data feed that your system generates at the same time every night. Business events trigger use cases in business applications. See Chapter 12, “A picture is worth 1024 words,” for more information.

Hold elicitation interviews Interviews can be performed one-on-one or with a small group of stakeholders. They are an effective way to elicit requirements without taking too much stakeholder time because you meet with people to discuss only the specific requirements that are important to them. Interviews are helpful to separately elicit requirements from people in preparation for workshops where those people come together to resolve any conflicts. See Chapter 7 for more information.

Hold facilitated elicitation workshops Facilitated requirements-elicitation workshops that permit collaboration between analysts and customers are a powerful way to explore user needs and to draft requirements documents (Gottesdiener 2002). Such workshops are sometimes called Joint Application Design, or JAD, sessions (Wood and Silver 1995). See Chapter 7 for more information.

Observe users performing their jobs Watching users perform their business tasks establishes a context for their potential use of a new application. Simple process flow diagrams can depict the steps and decisions involved and show how different user groups interact. Documenting the business process flow will help you identify requirements for a solution that’s intended to support that process. See Chapter 7 for more information.

Distribute questionnaires Questionnaires are a way to survey large groups of users to determine what they need. Questionnaires are useful with any large user population but are particularly helpful with distributed groups. If questions are well written, questionnaires can help you quickly determine analytical information about needs. Additional elicitation efforts can then be focused according to the questionnaire results. See Chapter 7 for more information.

Perform document analysis Existing documentation can help reveal how systems currently work or what they are supposed to do. Documentation includes any written information about current systems, business processes, requirements specifications, competitor research, and COTS (commercial off-the-shelf) package user manuals. Reviewing and analyzing the documents can help identify functionality that needs to remain, functionality that isn’t used, how people do their jobs currently, what competitors offer, and what vendors say their software should do. See Chapter 7 for more information.

Examine problem reports of current systems for requirement ideas Problem reports and enhancement requests from users provide a rich source of ideas for capabilities to include in a later release or in a new product. Help desk and support staff can provide valuable input into the requirements for future development work.

Reuse existing requirements If customers request functionality similar to that already present in an existing product, see whether the requirements (and the customers!) are flexible enough to permit reusing or adapting the existing software components. Projects often can reuse those requirements that comply with an organization’s business rules, such as security requirements, and requirements that conform to government regulations, such as accessibility requirements. Other good candidates for reuse include glossaries, data models and definitions, stakeholder profiles, user class descriptions, and personas. See Chapter 18, “Requirements reuse,” for more information.

Good practices: Requirements analysis

Requirements analysis involves refining the requirements to ensure that all stakeholders understand them and scrutinizing them for errors, omissions, and other deficiencies. Analysis includes decomposing high-level requirements into appropriate levels of detail, building prototypes, evaluating feasibility, and negotiating priorities. The goal is to develop requirements of sufficient quality and precision that managers can construct realistic project estimates and technical staff can proceed with design, construction, and testing.

It is very valuable to represent some of the requirements in multiple ways—for example, in both textual and visual forms, or in the forms of both requirements and tests (Wiegiers 2006). These different views will reveal insights and problems that no single view can provide. Multiple views also help all stakeholders arrive at a common understanding—a shared vision—of what they will have when the product is delivered.

Model the application environment The context diagram is a simple analysis model that shows how the new system fits into its environment. It defines the boundaries and interfaces between the system being developed and external entities, such as users, hardware devices, and other systems. An ecosystem map shows the various systems in the solution space that interact with each other and the nature of their interconnections (Beatty and Chen 2012). See Chapter 5 for more information.

Create user interface and technical prototypes When developers or users aren't certain about the requirements, construct a prototype—a partial, possible, or preliminary implementation—to make the concepts and possibilities more tangible. Prototypes allow developers and users to achieve a mutual understanding of the problem being solved, as well as helping to validate requirements. See Chapter 15, “Risk reduction through prototyping,” for more information.

Analyze requirement feasibility The BA should work with developers to evaluate the feasibility of implementing each requirement at acceptable cost and performance in the intended operating environment. This allows stakeholders to understand the risks associated with implementing each requirement, including conflicts and dependencies with other requirements, dependencies on external factors, and technical obstacles. Requirements that are technically infeasible or overly expensive to implement can perhaps be simplified and still contribute to achieving the project's business objectives.

Prioritize the requirements It's important to prioritize requirements to ensure that the team implements the highest value or most timely functionality first. Apply an analytical approach to determine the relative implementation priority of product features, use cases, user stories, or functional requirements. Based on priority, determine which release or increment will contain each feature or set of requirements. Adjust priorities throughout the project as new requirements are proposed and as customer needs, market conditions, and business goals evolve. See Chapter 16, “First things first: Setting requirements priorities,” for more information.

Create a data dictionary Definitions of the data items and structures associated with the system reside in the data dictionary. This enables everyone working on the project to use consistent data definitions. As requirements are developed, the data dictionary should define data items from the

problem domain to facilitate communication between the customers and the development team. See Chapter 13, “Specifying data requirements,” for more information.

Model the requirements An analysis model is a diagram that depicts requirements visually, in contrast to the textual representation of a list of functional requirements. Models can reveal incorrect, inconsistent, missing, and superfluous requirements. Such models include data flow diagrams, entity-relationship diagrams, state-transition diagrams, state tables, dialog maps, decision trees, and others (Beatty and Chen 2012). See Chapters 5, 12, and 13 for more information about modeling.

Analyze interfaces between your system and the outside world All software systems have connections to other parts of the world through external interfaces. Information systems have user interfaces and often exchange data with other software systems. Embedded systems involve interconnections between software and hardware components. Network-connected applications have communication interfaces. Analyzing these helps make sure that your application will fit smoothly into its environment. See Chapter 10, “Documenting the requirements,” for more information.

Allocate requirements to subsystems The requirements for a complex product that contains multiple subsystems must be apportioned among the various software, hardware, and human subsystems and components. An example of such a product is an access system to a secure building that includes magnetic or optical badges, scanners, video cameras and recorders, door locks, and human guards. See Chapter 26, “Embedded and other real-time systems projects,” for more information.

Good practices: Requirements specification

The essence of requirements specification is to document requirements of different types in a consistent, accessible, and reviewable way that is readily understandable by the intended audiences. You can record the business requirements in a vision and scope document. User requirements typically are represented in the form of use cases or user stories. Detailed software functional and nonfunctional requirements are recorded in a software requirements specification (SRS) or an alternative repository, such as a requirements management tool.

Adopt requirement document templates Adopt standard templates for documenting requirements in your organization, such as the vision and scope document template in Chapter 5, the use case template in Chapter 8, and the SRS template in Chapter 10. The templates provide a consistent structure for recording various groups of requirements-related information. Even if you don’t store the requirements in traditional document form, the template will remind you of the various kinds of requirements information to explore and record.

Identify requirement origins To ensure that all stakeholders know why every requirement is needed, trace each one back to its origin. This might be a use case or some other customer input, a high-level system requirement, or a business rule. Recording the stakeholders who are affected by each requirement tells you whom to contact when a change is requested. Requirement origins can be identified through traceability links or by defining a requirement attribute for this purpose. See Chapter 27, “Requirements management practices,” for more information on requirement attributes.

Uniquely label each requirement Define a convention for giving each requirement a unique identifying label. The convention must be robust enough to withstand additions, deletions, and changes made in the requirements over time. Labeling the requirements permits requirements traceability and the recording of changes made. See Chapter 10 for more information.

Record business rules Business rules include corporate policies, government regulations, standards, and computational algorithms. Document your business rules separately from a project's requirements because they typically have an existence beyond the scope of a specific project. That is, treat business rules as an enterprise-level asset, not a project-level asset. Some rules will lead to functional requirements that enforce them, so define traceability links between those requirements and the corresponding rules. See Chapter 9, "Playing by the rules," for more information.

Specify nonfunctional requirements It's possible to implement a solution that does exactly what it's supposed to do but does not satisfy the users' quality expectations. To avoid that problem, you need to go beyond the functionality discussion to understand the quality characteristics that are important to success. These characteristics include performance, reliability, usability, modifiability, and many others. Customer input on the relative importance of these quality attributes lets the developer make appropriate design decisions. Also, specify external interface requirements, design and implementation constraints, internationalization issues, and other nonfunctional requirements. See Chapter 14, "Beyond functionality," for more information.

Good practices: Requirements validation

Validation ensures that the requirements are correct, demonstrate the desired quality characteristics, and will satisfy customer needs. Requirements that seem fine when you read them might turn out to have ambiguities and gaps when developers try to work with them. You must correct these problems if the requirements are to serve as a reliable foundation for design and for final system testing and user acceptance testing. Chapter 17, "Validating the requirements," discusses this topic further.

Review the requirements Peer review of requirements, particularly the type of rigorous review called inspection, is one of the highest-value software quality practices available (Wiegers 2002). Assemble a small team of reviewers who represent different perspectives (such as analyst, customer, developer, and tester), and carefully examine the written requirements, analysis models, and related information for defects. Informal preliminary reviews during requirements development are also valuable. It's important to train the team members in how to perform effective requirements reviews and to adopt a review process for your organization. See Chapter 17 for more information.

Test the requirements Tests constitute an alternative view of the requirements. Writing tests requires you to think about how to tell if the expected functionality was correctly implemented. Derive tests from the user requirements to document the expected behavior of the product under specified conditions. Walk through the tests with customers to ensure that they reflect user expectations. Map the tests to the functional requirements to make sure that no requirements have been overlooked and that all have corresponding tests. Use the tests to verify the correctness of analysis models and prototypes. Agile projects often create acceptance tests in lieu of detailed functional requirements. See Chapter 17 for more information.

Define acceptance criteria Ask users to describe how they will determine whether the solution meets their needs and is fit for use. Acceptance criteria include a combination of the software passing a defined set of acceptance tests based on user requirements, demonstrating satisfaction of specific nonfunctional requirements, tracking open defects and issues, having infrastructure and training in place for a successful rollout, and more. See Chapter 17 for more information.

Simulate the requirements Commercial tools are available that allow a project team to simulate a proposed system either in place of or to augment written requirements specifications. Simulation takes prototyping to the next level, by letting BAs work with users to rapidly build executable mock-ups of a system. Users can interact with the simulated system to validate requirements and make design choices, making the requirements come to life before they are cast into the concrete of code. Simulation is not a substitute for thoughtful requirements elicitation and analysis, but it does provide a powerful supplement.

Good practices: Requirements management

After you have the initial requirements for a body of work in hand, you must cope with the inevitable changes that customers, managers, marketing, the development team, and others request during development. Effective change management demands a process for proposing changes, evaluating their potential cost and impact on the project, and making sure that appropriate stakeholders make sensible business decisions about which proposed changes to incorporate.

Well-established configuration management practices are a prerequisite for effective requirements management. The same version control tools that you use to control your code base can manage your requirements documents. Even better, store requirements in a requirements management tool, which provides many capabilities to perform these practices.

Establish a requirements change control process Rather than stifling change or hoping changes don't happen, accept the fact that they will and establish a mechanism to prevent rampant changes from causing chaos. Your change process should define how requirements changes are proposed, analyzed, and resolved. Manage all proposed changes through this process. Defect-tracking tools can support the change control process. Charter a small group of project stakeholders as a change control board (CCB) to evaluate proposed requirements changes, decide which ones to accept, and set implementation priorities or target releases. See Chapter 28, "Change happens," for more information.

Perform impact analysis on requirements changes Impact analysis is an important element of the change process that helps the CCB make informed business decisions. Evaluate each proposed requirement change to assess the effect it will have on the project. Use the requirements traceability matrix to identify the other requirements, design elements, source code, and tests that you might need to modify. Identify the tasks required to implement the change and estimate the effort needed to perform those tasks. See Chapter 28 for more information.

Establish baselines and control versions of requirements sets A baseline defines a set of agreed-upon requirements, typically for a specific release or iteration. After the requirements have been baselined, changes should be made only through the project's change control process. Give

every version of the requirements specification a unique identifier to avoid confusion between drafts and baselines and between previous and current versions. See Chapter 2, “Requirements from the customer’s perspective,” and Chapter 27 for more information.

Maintain a history of requirements changes Retain a history of the changes made to individual requirements. Sometimes you need to revert to an earlier version of a requirement or want to know how a requirement came to be in its current form. Record the dates that requirements were changed, the changes that were made, who made each change, and why. A version control tool or requirements management tool can help with these tasks.

Track the status of each requirement Establish a repository with one record for each discrete requirement of any type that affects implementation. Store key attributes about each requirement, including its status (such as proposed, approved, implemented, or verified), so you can monitor the number of requirements in each status category at any time. Tracking the status of each requirement as it moves through development and system testing provides insight into overall project status. See Chapter 27 for more information.

Track requirements issues When busy people are working on a complex project, it’s easy to lose sight of the many issues that arise, including questions about requirements that need resolution, gaps to eradicate, and issues arising from requirements reviews. Issue-tracking tools can keep these items from falling through the cracks. Assign a single owner to each issue. Monitor the status of requirement issues to determine the overall state of the requirements. See Chapter 27 for more information.

Maintain a requirements traceability matrix It’s often valuable—and sometimes required—to assemble a set of links that connect each functional requirement to the design and code elements that implement it and the tests that verify it. Such a *requirements traceability matrix* is helpful for confirming that all requirements are implemented and verified. It’s also useful during maintenance when a requirement has to be modified. The requirements traceability matrix can also connect functional requirements to the higher-level requirements from which they were derived and to other related requirements. Populate this matrix during development, not at the end. Tool support is essential on all but the smallest projects. See Chapter 29, “Links in the requirements chain,” for more information.

Use a requirements management tool Commercial requirements management tools let you store various types of requirements in a database. Such tools help you implement and automate many of the other requirements management practices described in this section. See Chapter 30, “Tools for requirements engineering,” for more information.

Good practices: Knowledge

Various team members might perform the role of business analyst on a given project, but few software practitioners receive formal training in requirements engineering. Business analysis is a specialized and challenging role, with its own body of knowledge (IIBA 2009). As with all technical disciplines, there is no substitute for experience. It isn’t reasonable to expect all people to be

instinctively competent at the communication-intensive tasks of requirements engineering. Training can increase the proficiency and comfort level of those who serve as analysts, but it can't compensate for absent interpersonal skills or a lack of interest in the role.

Train business analysts All team members who will perform BA tasks, whether they have the job title "business analyst" or not, should receive training in requirements engineering. Business analyst specialists need several days of training in the diverse activities that BAs typically perform. This will give them a solid foundation on which to build through their own experiences and advanced training. In addition to having an extensive tool kit of techniques, the skilled analyst is patient and well organized, has effective interpersonal and communication skills, and understands the application domain. See Chapter 4, "The business analyst," for more information about this important role.

Educate stakeholders about requirements The most effective requirements training classes have an audience that spans multiple project functional areas, not just BAs. Users who will participate in software development should receive one or two days of education about requirements so they understand terminology, key concepts and practices, and why this is such an important contributor to project success. Development managers and customer managers will also find this information useful. Bringing together the various stakeholders for a class on software requirements can be an effective team-building activity. All parties will better appreciate the challenges their counterparts face and what the participants require from each other for the whole team to succeed. Some users who have attended our requirements classes have said that they came away with more sympathy for the software developers.

Educate developers about the application domain To help give developers a basic understanding of the application domain, arrange a seminar on the customer's business activities, terminology, and objectives for the product being created. This can reduce confusion, miscommunication, and rework down the road. "Day-in-the-life" experiences in which developers accompany users to see how they perform their jobs are sound investments. You might also match each developer with a "user buddy" for the life of the project to translate jargon and explain business concepts. The product champion could play this role, as described in Chapter 6.

Define a requirements engineering process Document the steps your organization follows to elicit, analyze, specify, validate, and manage requirements. Providing guidance on how to perform the key steps will help analysts do a consistently good job. It will also make it easier to plan each project's requirements development and management tasks, schedule, and required resources. The project manager should incorporate requirements activities as discrete tasks in the project plan. See Chapter 31, "Improving your requirements processes," for more information.

Create a glossary A glossary that defines specialized terms from the application domain will minimize misunderstandings. Include synonyms, acronyms or abbreviations, terms that can have multiple meanings, and terms that have both domain-specific and everyday meanings. A glossary could be a reusable enterprise-level asset. Developing a glossary could be an activity for new team members, because they will be the ones most confused by the unfamiliar terminology. See Chapter 10 for more information on the glossary.

Good practices: Project management

Software project management approaches are tightly coupled to a project's requirements processes. The project manager should base project schedules, resources, and commitments on the requirements that are to be implemented. An alternative strategy is to timebox development cycles, such that the team estimates the scope of the work they can fit into an iteration of fixed duration. This is the approach taken by agile development projects. Scope is regarded as negotiable within the schedule. This transforms scope creep into "scope choice"—the product owner can ask for anything and as much as he wants, but he must prioritize it, and the team quits developing when they run out of time. Then the team plans a subsequent release for the remaining requirements.

Select an appropriate software development life cycle Your organization should define several development life cycles that are appropriate for various types of projects and different degrees of requirements uncertainty (Boehm and Turner 2004). Each project manager should select and adapt the life cycle that best suits her project. Include requirements activities in your life cycle definitions. When possible, specify and implement sets of functionality incrementally so that you can deliver useful software to the customer as early as possible (Larman 2004; Schwaber 2004; Leffingwell 2011).

Plan requirements approach Each project team should plan how it will handle its requirements development and management activities. An elicitation plan helps ensure that you identify and obtain input from appropriate stakeholders at the right stages of the project using the most appropriate techniques. The BA and project manager should work together to ensure that tasks and deliverables related to requirements engineering appear in the project management plan. See Chapter 7 for more information.

Estimate requirements effort Stakeholders often want to know how long it's going to take to develop the requirements for a project and what percentage of their total effort should be devoted to requirements development and management. Naturally, this depends on many factors. Consider the factors that would indicate that you should spend either more or less time than average to ensure the requirements lay a solid foundation for development (Wiegiers 2006). See Chapter 19, "Beyond requirements development," for more information.

Base project plans on requirements Develop plans and schedules for your project iteratively as the scope and detailed requirements become clear. Begin by estimating the effort needed to develop the user requirements from the initial product vision and project scope. Early cost and schedule estimates based on fuzzy requirements will be highly uncertain, but you can improve the estimates as your understanding of the requirements improves. On agile projects, the timeboxed nature of iterations means that planning involves adjusting the scope to fit within the fixed schedule and resource constraints. See Chapter 19, "Beyond requirements development," and Chapter 20, "Agile projects," for more information.

Identify requirements decision makers Software development involves making many decisions. Conflicting user inputs must be resolved, commercial package components must be selected, change requests must be evaluated, and on and on. Because so many decisions involve requirements issues, it's essential for the project team to identify and empower its requirements decision makers, preferably before they confront their first significant decision. See Chapter 2 for more information.

Renegotiate project commitments when requirements change A project team makes commitments to deliver specific sets of requirements within a particular schedule and budget. As you incorporate new requirements into the project, evaluate whether you can still achieve the current commitments with the available resources. If not, communicate the project realities to management and negotiate new, realistically achievable commitments (Wiegers 2007; Fisher, Ury, and Patton 2011). You might also need to renegotiate commitments as requirements evolve from their fuzzy beginnings with initial implementation estimates to clear, validated requirements.

Analyze, document, and manage requirements-related risks Unanticipated events and conditions can wreak havoc on an unprepared project. Identify and document risks related to requirements as part of the project's risk-management activities. Brainstorm approaches to mitigate or prevent these risks, implement the mitigation actions, and track their progress and effectiveness. See Chapter 32, "Software requirements and risk management," for more information.

Track the effort spent on requirements To improve your ability to estimate the resources needed for requirements work on future projects, record the effort your team expends on requirements development and management activities (Wiegers 2006). Monitor the effect that your requirements activities have on the project to help judge the return on your investment in requirements engineering. See Chapter 27 for more information.

Review lessons learned regarding requirements on other projects A learning organization conducts periodic retrospectives to collect lessons learned from completed projects or from earlier iterations of the current project (Kerth 2001; Derby and Larsen 2006; Wiegers 2007). Studying the lessons learned from previous requirements experiences can help project managers and business analysts steer a more confident course in the future.

Getting started with new practices

Table 3-2 groups the requirements engineering good practices described in this chapter by the relative value they can contribute to most projects and their relative difficulty of implementation. These classifications are not absolute; your experiences might be different. Although all the practices can be beneficial, you might begin with those practices that have a high impact on project success and are relatively easy to implement.

TABLE 3-2 Implementing requirements engineering good practices

Value	Difficulty		
	High	Medium	Low
High	■ Define a requirements engineering process ■ Base plans on requirements ■ Renegotiate commitments	■ Train business analysts ■ Plan requirements approach ■ Select product champions ■ Identify user requirements ■ Hold elicitation interviews ■ Specify nonfunctional requirements ■ Prioritize requirements ■ Define vision and scope ■ Establish a change control process ■ Review the requirements ■ Allocate requirements to subsystems ■ Use a requirements management tool ■ Record business rules	■ Educate developers about application domain ■ Adopt requirement document templates ■ Identify user classes ■ Model the application environment ■ Identify requirement origins ■ Establish baselines and control versions of requirements sets ■ Identify requirements decision makers
Medium	■ Maintain a requirements traceability matrix ■ Hold facilitated elicitation workshops ■ Estimate requirements effort ■ Reuse existing requirements	■ Educate stakeholders about requirements ■ Conduct focus groups ■ Create prototypes ■ Analyze feasibility ■ Define acceptance criteria ■ Model the requirements ■ Analyze interfaces ■ Perform change impact analysis ■ Select an appropriate life cycle ■ Identify system events and responses ■ Manage requirements risks ■ Review past lessons learned ■ Track requirements effort	■ Create a data dictionary ■ Observe users performing their jobs ■ Test the requirements ■ Track requirements status ■ Perform document analysis ■ Track requirements issues ■ Uniquely label each requirement ■ Create a glossary
Low		■ Distribute questionnaires ■ Maintain change history ■ Simulate the requirements	■ Examine problem reports

Don't try to apply all of these techniques on your next project. Instead, think of these good practices as new items for your requirements tool kit. You can begin to use certain practices, such as those dealing with change management, no matter where your project is in its development cycle. Elicitation practices will be more useful when you begin the next project or iteration. Still others might not fit your current project, organizational culture, or resource availability. Chapter 31 and Appendix A describe ways to evaluate your organization's current requirements engineering practices. Chapter 31 will help you devise a road map for implementing selected improvements in your requirements process based on the practices described in this chapter. Incorporate the adoption of new requirements techniques into your organization's software process improvement activities, relying on change leadership to facilitate the piloting, rollout, and adoption of better practices. Just make sure that each of your development teams tries something new and better at each opportunity.



Next steps

- Go back to the requirements-related problems you identified from the Next Steps in Chapter 1. Identify good practices from this chapter that might help with each problem you identified. Group the practices into high, medium, and low impact in your organization. Identify any barriers to implementing each practice in your organization or culture. Who can help you break down those barriers? Can you pick one activity to begin performing better than you already are?
- Determine how you would assess the benefits from the practices that you think would be most valuable. Would you find fewer requirements defects late in the game, reduce unnecessary rework, better meet project schedules, achieve higher customer satisfaction or product sales, or enjoy other advantages?
- List all the requirements good practices you identified in the first step. For each, indicate your project team's current level of capability: expert, proficient, novice, or unfamiliar. If your team is not at least proficient in any of those practices, ask someone on your project to learn more about the practice and to share what he learns with the rest of the team.

